FAST UNIVERSITY
BSE-5A

AI PROJECT
Phase-1

TEAM MEMBERS

Eman Fatima	(24L-3008)
Abdullah Tahir	(24L-3061)
Shahzaib Zia	(24L-3057)
Rabia	(24L-3093)

PROJECT DELIVERABLE

AI-POWERED E-COMMERCE ANALYTICS AND CONVERSATIONAL AGENT SYSTEM

1. Problem Statement

Most small e-commerce sellers, especially fashion sellers, run their sales over WhatsApp and Instagram. Customers message them directly to ask about stock, price, and delivery, and staff reply manually. This causes a few real problems. Replies get delayed, especially after hours. Different staff members give different answers about price or delivery charges. Order details get mistyped from chat, wrong size or wrong color. Stock isn't updated in real time, so items get oversold. And staff end up spending most of their time answering the same repetitive questions instead of doing anything more useful.

A simple rule-based chatbot can't really fix this, because customer messages are messy, mixed Roman Urdu/English, and often depend on something said earlier in the conversation. For example: “Bhai wo black wala jo kal story mein tha, medium mein hai?” Even if a bot understands a message like this, it still has to safely complete a real action, like placing an order, and a plain chatbot or an ungrounded LLM can easily get these details wrong.

So the actual problem we're trying to solve is this: can a conversational AI agent take normal, mixed-language customer chat and turn it into correct, policy-compliant actions such as checking stock, creating an order, checking order status, or handling an exchange, while also knowing when it should not act on its own and should hand off to a human instead? As a smaller, secondary goal, we also want to look at whether the same conversation data could later help the store owner understand customer behaviour, like which questions come up the most or what products people usually ask about.

2. Dataset

We're using two data sources for this project.

- MultiWOZ, a well-known multi-domain, task-oriented dialogue dataset. We're using it to help build and test the multi-turn dialogue tracking and entity extraction part of the agent (product, size, color, quantity), since its structure is close to what we need for order-taking conversations.
- A custom dataset that we're building ourselves. This includes a sample product catalog (products, sizes, colors, prices, stock), a policy document (delivery charges, COD rules, exchange and return policy), and a set of Roman Urdu/English customer conversations we're writing and generating for the common cases: checking stock, asking for a recommendation within a budget, placing a multi-turn order, checking order status, and asking for an exchange. This is what grounds the RAG layer so the AI doesn't guess at a price or a policy that doesn't actually exist.

MultiWOZ dataset link: https://github.com/budzianowski/multiwoz

3. Reference Paper

Balaji, S., Mishra, P., Sachdeva, A., and Agrawal, S. (2026). “Beyond IVR: Benchmarking Customer Support LLM Agents for Business-Adherence.” EACL 2026, Industry Track, pages 193 to 208.

Paper link: https://aclanthology.org/2026.eacl-industry.15/

This paper introduces JourneyBench, a benchmark that checks whether an LLM support agent actually follows business policy step by step, not just whether it completes the task. They propose a metric called the User Journey Coverage Score for this. That's basically close to what we're trying to measure in our own project too, so it's a strong reference point for our guardrail layer, our escalation logic, and how we plan to evaluate the agent later on (policy adherence, task completion, and correctly knowing when to hand off to a human).

4. Scope of the Project

In scope for this phase:

- A simulated chat interface (WhatsApp/website style) that understands Roman Urdu/English.
- Pulling product, size, color, quantity, and delivery location out of normal conversation, including references to something said earlier (e.g. “2 same bhej do aur ek white”).
- A RAG layer grounded in the catalog, current stock, and the policy documents.
- Real backend actions through tool calls: check_stock(), reserve_order(), create_order(), get_order_status(), check_delivery_policy().
- A guardrail layer that checks confidence and business rules before letting the AI act on its own, for example, no exchange on sale items.
- Human handoff for anything ambiguous, low-confidence, or outside policy, like a damaged-item refund, with full context passed on to the owner.
- A basic dashboard showing conversations handled, orders placed, escalations, and an error log.
- Measuring hallucination rate, order accuracy, how the system handles multiple customers ordering the last few units at once, escalation accuracy, and task completion rate.

Out of scope for now:

- A full analytics/BI layer for the owner, like “why did sales drop this week”. That's a possible future extension, not part of this phase.
- Actual Instagram/WhatsApp API integration or voice. We're working with a simulated chat interface for now.
- Real payment gateway or courier/logistics integration.

The customer is the main user of this system. The owner only gets involved when something actually needs their attention. That's the main difference between this and a tool the owner would use directly to manage their store.